Chat System — Phase 2

Student: Lowry Zhao
Student ID: S5226106

Git Repository Organisation

Repository Structure

- **client**： Used to store Angular frontend components, the UI components (components), data models (models), Business logic (services), Angular routes (app.routes.ts) and E2E testing(cypress).
- use `ng serve` to start.
- **server**： Used to store backend files, mainly including Rest API end points (routes), Business logic (controllers), MongoDB connection (db.js), Main server + Socket.IO (server.js), Uploaded avatars/images (uploads) and Jest unit tests (test).
- use `node sever.js` to start.
- **Git**: Because only one main branch was used, uploading to GitHub only used `git add .`, `git commit -m " "`, `git push`.

Version Control Strategy

- `main` — stable release branch
- All code uploaded to Git is tested and validated locally before being uploaded to Github.
- Each commit message is descriptive (`video chat`, `fix send message`)
- `.gitignore` excludes `node_modules`, `.env`, build folders, and uploads.

Data Structures

Server-side (MongoDB Collections)

Collection	Fields	Description
`users`	`{ id, username, email, password, roles[], groups[], avatar }`	Stores users, roles (`super_admin`, `group_admin`, `user`), and avatar paths
`groups`	`{ id, name, admins[], members[], channels[] }`	Group structure with admins and members
`channels`	`{ id, name, groupId, members[] }`	Channels within groups
`messages`	`{ channelId, userId, username, avatar, message, imageUrl, timestamp }`	Chat messages with text or image

- **roles** are used for permission control (super add, group add, user).
- **Groups** is a list of group IDs that users join
- **Avatar** is the path to the avatar file, which is physically stored on server/uploads/avatars/, and the path is stored in the database.

- Groups and channels are associated through channels ID.
- Display the group name and number of members in the front-end **GroupListComponent**.
- **groupId** represents the group to which the channel belongs.
- **members** is a list of users who can access the channel.
- The front-end implements real-time channel joining and leaving (joinChannel/leaveChannel) through Socket.IO.

API Routes

Route	Method	Parameters	Return	Description
`/api/users/login`	POST	`{ username, password }`	`User object`	Authenticate login
`/api/users/register`	POST	`{ username, email, password }`	`User object`	Register user
`/api/users/update-avatar`	POST	`{ userId, avatarPath }`	`{ message, avatar }`	Update avatar
`/api/upload/avatar`	POST	FormData(`avatar`)	`{ path }`	Upload profile avatar
`/api/upload/image`	POST	FormData(`image`)	`{ path }`	Upload chat image
`/socket.io`	WebSocket	—	—	Real-time communication
`/peerjs`	PeerJS	—	—	Peer-to-peer video signaling

Client-Server Responsibilities

- **Server**: Handling data persistence (MongoDB), authentication (JWT), file upload (Express multer), and real-time broadcasting (Socket. IO). Provide REST API to return JSON and serve static files (/uploads).
- **Client**: Manage UI rendering (Angular components), user interaction (forms), local state (NgRx or services), HTTP requests (FHIR), and WebSocket subscriptions. This ensures that the server is stateless and the client is responsive.

Angular Architecture

Components

- LoginComponent: Process login/registration forms.
- GroupListComponent: Display user group list
- ChannelListComponent: Channel selection, Socket.IO subscription join/leave
- ChatComponent: Message input/display, processing and sending, and image preview.
- VideoChatComponent: WebRTC video streaming integration.
- ProfileComponent: Upload and update avatars.

Services

- AuthService: Login/registration, token storage (localStorage).
- ChatService: Socket.IO events (sendMessage, onMessage), message history loading.
- UploadService: File upload (FHIR POST FormData).

Models

- src/app/models/user.model.ts: TypeScript interface (as shown in the above data structure).

Routes (app.routes.ts)

const routes: Routes = [{ path: 'login', component: LoginComponent }, { path: 'groups', component: GroupListComponent }, { path: 'groups/:groupId/channels/:channelId', component: ChatComponent }, { path: 'video/:peerId', component: VideoChatComponent }];

Client–Server Interaction

Overview

The communication between the **Angular client** and the **Express/MongoDB server** follows a hybrid architecture combining:

- **RESTful APIs** (for CRUD operations & file uploads)
- **Socket.IO WebSockets** (for real-time chat and presence updates)
- **PeerJS WebRTC signaling** (for video chat connections)

Both client and server maintain synchronized states via API responses and event broadcasting.

Server-Side File & Variable Changes

Action	Server-Side Process	Files / Variables Updated
User Registration	A new document is inserted into the `users` MongoDB collection with default avatar path `/uploads/default-avatar.png`.	`server/controllers/userController.js` → `users.insertOne()`
Login	Server validates credentials and returns user object as JSON.	`server/controllers/userController.js`
Avatar Upload	Image is saved in `/uploads/avatars/`. The file path is stored in MongoDB (`users.avatar`).	`server/routes/uploadRoutes.js` + `users.updateOne()`
Send Chat Message	Server receives Socket.IO `chatMessage` event, stores message object (`message`, `imageUrl`, `timestamp`) in MongoDB `messages` collection.	`server/server.js` (Socket.IO section) + `db.collection('messages').insertOne()`

Action	Server-Side Process	Files / Variables Updated
Send Chat Image	File is saved in `/uploads/chat/` directory. File path stored in `messages.imageUrl`.	`server/routes/uploadRoutes.js`
Join / Leave Channel	Socket joins or leaves a room (`socket.join(channelId)`), updates `io.to(channelId)` broadcast list.	`server/server.js` (Socket.IO)
Video Chat	PeerJS server manages signaling between connected peers. No DB change.	`server/server.js` (`ExpressPeerServer`)

Angular Component Updates

User Action	Angular Component	Server Response	UI Update
Login	`LoginComponent`	JSON user object	Stores in `localStorage`, redirects to `/groups`
View Groups	`GroupListComponent`	GET `/api/groups/user/:id`	List of groups rendered dynamically
Select Channel	`ChannelListComponent`	Socket emits `joinChannel`	Loads last 20 messages via `chatHistory` event
Send Message	`ChatComponent`	Emits `chatMessage` via Socket.IO	Message appended to chat window instantly
Upload Image	`ChatComponent`	POST `/api/upload/image` → `{ path }`	Image preview shown in chat bubble
Update Avatar	`ProfileComponent`	POST `/api/upload/avatar` + `/api/users/update-avatar`	Profile avatar image updates immediately
Start Video Chat	`VideoChatComponent`	PeerJS connection established	Video feed streams into `<video>` elements

Example

Example — Send Message in chat:

1. `ChatComponent.send()` → emits `chatMessage` with `{channelId, userId, username, message}`
2. `server.js` catches event:

```
io.on('connection', socket => {
  socket.on('chatMessage', async data => {
```

```
    await db.collection('messages').insertOne(data);
    io.to(data.channelId).emit('chatMessage', data);
  });
});
```

3. Angular `ChatService.onMessage()` listener receives broadcast.